Trajectory Generation for Multiprocess Robotic Tasks Based on Nested Dual-Memory Deep Deterministic Policy Gradient

Fengkang Ying, Huashan Liu, *Member, IEEE*, Rongxin Jiang, and Xin Yin

***Abstract*—Though there are extensive works on deep reinforcement learning (DRL) for robotics, sequential trajectory generation for multiprocess robotic tasks based on DRL is yet to be explored. In this article, the multiprocess task is formulated as a Markov decision process, and a nested dual-memory deep deterministic policy gradient algorithm with dynamic criteria is proposed, to generalize the traditional trajectory planning with predefined target point into a trajectory exploration problem aiming at a target area without solving inverse kinematics. First, a dual-memory architecture with local-to-global strategy is introduced to enhance the performance. Second, a novel nested architecture is proposed to generate sequential trajectory segments successively and asynchronously for the multiprocess task. Third, a compound reward system is designed and a weight coefficient matrix is adopted to balance the position control and the orientation control based on Tait–Bryan angles. In addition, a virtual twin system is established to promote the training efficiency, where the trajectory generated in simulation can be directly applied to the real physical platform. Finally, experimental results on both simulated and real-world applications have verified the performance of the proposed approach.**

***Index Terms*—Compound reward system, dual-memory deep deterministic policy gradient, nested architecture, trajectory generation, virtual twin system.**

I. Introduction

IN RECENT years, robots have been playing an increasingly significant role in industrial manufacturing and processing for their high applicability, efficiency, and accuracy of dealing with complex tasks in various scenarios [1]. Robotic pick-and-place assembly is one of the most common tasks applied to industrial systems. For a specific scene, the end-effector (such as gripper) of the robot moves through a preplanned trajectory from one position to another to pick up and place finished or nonfinished assembly products [2]. However, such automatic pick-and-place task still faces numerous challenges in complex scenarios, such as changeable tasks and assembly objects and unstructured environments [3].

Traditional trajectory planning methods usually need to know in advance the position and posture of the robot at the start and end points, as well as specific methods for solving inverse kinematics (IK) and planning [2], [4]. This is particularly complicated for robots with redundant degrees of freedom (DoFs) that have infinite IK solutions, and extensive works have been contributed to solving the IK and motion planning problems for manipulators with redundant DoFs in [5] and [4]. However, these methods are dedicated to a certain type of redundant robots and are not feasible for arbitrary DoFs, and when even slight changes occur in the start point and/or the end point, replanning and recalculation are required. Thus, the algorithms mentioned above are challenging in real-world automatic assembly tasks as considerable efforts are required to generate feasible trajectories based on models of both the manipulator and the environment [6], [7]. To solve such problems, a more generalized method that can generate feasible trajectories without IK and is suitable for manipulators with arbitrary DoFs is expected.

Unlike robots controlled by traditional systems that usually rely on model parameters and can only perform fixed actions, humans can handle pick-and-place tasks flexibly and become skilled through observing and learning [8]. Recently, deep reinforcement learning (DRL), which uses deep learning as a nonlinear approximator, has begun to arouse interests of looking for the feasibility to make robots intelligent and capable to learn and optimize actions for robotic assembly tasks [9], [10]. For example, in [11] and [12], deep Q-network algorithm has been utilized to make the manipulator push and grasp cluttered objects in both simulated and realistic environments. A curriculum learning-based method has been proposed in [13] for picking assignments. Deep proximal policy optimization algorithm has been adopted to implement autonomous obstacle avoidance for manipulators in [14]. Decentralized control for mobile robots based on asynchronous advantage actor-critic (A3C) algorithm has been reported in [15]. Deep deterministic policy gradient (DDPG) algorithms have been utilized to control a 6-DoF manipulator to make the end-effector reach the target

Manuscript received 26 September 2021; revised 1 January 2022; accepted 15 March 2022. Date of publication 13 April 2022; date of current version 14 December 2022. Recommended by Technical Editor S. Jeon and Senior Editor Hong Qiao. This work was supported in part by the Natural Science Foundation of Shanghai under Grant 21ZR1401100 and in part by Shanghai Rising-Star Program under Grant 19QA1400400. *(Corresponding author: Huashan Liu.)*

Fengkang Ying, Rongxin Jiang, and Xin Yin are with the College of Information Science and Technology, Donghua University, Shanghai 201620, China (e-mail: fkying@mail.dhu.edu.cn; rxjiang@mail.dhu.edu.cn; xinyin@mail.dhu.edu.cn).

Huashan Liu is with the College of Information Science and Technology, Donghua University, Shanghai 201620, China, and also with the Engineering Research Center of Digitized Textile and Apparel Technology, Ministry of Education, Donghua University, Shanghai 201620, China (e-mail: hsliu@dhu.edu.cn).

This article has supplementary material provided by the authors and color versions of one or more figures available at https://doi.org/10.1109/TMECH.2022.3160605.

Digital Object Identifier 10.1109/TMECH.2022.3160605

1083-4435 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 25,2026 at 09:16:33 UTC from IEEE Xplore. Restrictions apply.

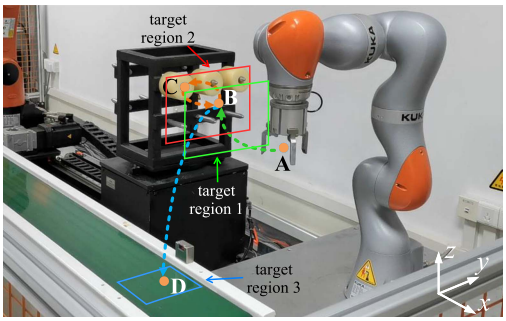


Fig. 1. Physical scenario of the pick-and-place assembly. A is the initial position of the tool center point (TCP). B, C, and D are random points in target regions 1, 2, and 3, respectively.

point in [16] and perform peg-in-hole assembly in [17]. In [18], DDPG, twin-delayed DDPG (TD3) and soft actor-critic (SAC) algorithms have been used to control soft robots.

Particularly, it is worth mentioning that, DDPG algorithm [19] has been validated to be robust, stable, and effective in handling complex cases with continuous and high-dimensional action and state space [20]. However, there are still some drawbacks of DDPG in multiprocess scenarios like robotic pick-and-place assembly tasks and other similar kinds.

1) Significance of the experience data is neglected and may cause instability, especially in early training where there is little successful experience data. Though prioritized experience replay (PER) is able to find more valuable data samples under the environment with sparse rewards [21], the effect will be weakened when the rewards are dense enough.
2) In a mass of scenarios, robotic tasks have multiple processes, like pick-and-place assembly shown in Fig. 1, where some adjacent processes are not independent and connected in time order. However, only single trajectory segment is performed to execute tasks in most DRL approaches [11], [12], [16]. Though multi-DDPG algorithms have been reported [22]–[25], the DDPGs are activated simultaneously for parallel tasks. They are not feasible in generating sequential trajectory segments where each algorithm is dependent on time sequence.
3) In many DRL applications for manipulators, the translation and rotation of the tool center point (TCP) are taken as the action to change both the position and orientation [13], [17]. However, such applications depend on specific IK methods, and some tricks like using fingers to correct the orientation when closing up, or using suction cups are adopted [11], [12]. Besides, there is always a lack of a desired orientation reward in such works. Moreover, controlling the orientation by directly rotating the TCP will possibly cause gimbal lock. A feasible orientation control method that can avoid gimbal lock and how it cooperates with position control should be considered when designing reward functions.

To respond to the above discussions, in this article, we aim to handle the sequential trajectory segments generation problem without solving IK for multiprocess robotic tasks from a generalized perspective based on DDPG. The main contributions can be summarized as follows:

1) A dual-memory DDPG (DMDDPG) algorithm with local-to-global strategy is proposed for efficient policy learning. While the first memory utilizes the first-in-first-out (FIFO) principle when storing experience data, the second divides the data into two intervals according to rewards and performs a local-to-global strategy in training. The proposed dual-memory architecture can motivate the agent to acquire more successful experiences in the early training, and can promote a simple DDPG with single hidden layer to achieve almost the same effect as that with multiple hidden layers.
2) A nested DMDDPG (N-DMDDPG) architecture is proposed to generate trajectory segments that are dependent on time sequence for the multiprocess robotic tasks. Unlike the existing multiagent systems for parallel tasks [22]–[25], the proposed N-DMDDPG develops a multipolicy agent that can execute sequential tasks successively.
3) A compound reward system with a weight coefficient matrix is designed to integrate the orientation control based on Tait–Bryan angles [26] with the traditional position control. By combining the compound reward system with the nested architecture, the weight and importance of the orientation control can be flexibly tuned when generating different trajectory segments for multiprocess task.

In addition, a virtual twin system (VTS) is constructed to seamlessly apply policies trained in simulation to real-world applications and avoid redundant works.

The rest of this article is organized as follows. Section II describes the fundamentals of the proposed DMDDPG algorithm. The details of the nested structure are described in Section III. The composition of the compound reward system is formulated in Section IV. Simulated and realistic experiments are presented in Section V. Finally, Section VI concludes this article.

II. Fundamentals of Algorithm

In this section, the formulation of the multiprocess robotic task is defined and the basic principles of reinforcement learning are illustrated, then a single DMDDPG algorithm as the basis of the proposed N-DMDDPG approach is given, where its structure and training principle are presented.

A. Problem Formulation

For the applications of DRL in multiprocess robotic tasks like pick-and-place assembly, the prime target is to make the agent capable to find the optimal pick-and-place policy π^* through self-learning by interacting with the environment, in which both position and orientation should be considered.

This task can be described as a Markov decision process. Assume that at time step t, the agent obtains the current state s_t from the environment and takes an action a_t according to the policy π, thus the state s_t transforms to s_{t+1} and the agent gets a reward r_t as a feedback from the environment. In the training process, the agent optimizes the policy through maximizing the discounted future cumulative reward G_t, which is defined as

$$G_t = r_t + \rho r_{t+1} + \rho^2 r_{t+2} + \cdots + \rho^{n-t} r_n \tag{1}$$

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 25,2026 at 09:16:33 UTC from IEEE Xplore. Restrictions apply.

where r_t and r_n are the rewards of the current and final time step, $r_{t+i}(i = 1, 2, \cdots, n-t)$ is the future reward, and $\rho \in [0, 1]$ is a discount factor which decides the weight of future rewards.

As shown in Fig. 1, the training object in the pick-and-place task is the 7-DoF serial manipulator KUKA LBR iiwa 7 R800. The target is to generate an optimal trajectory for the manipulator to grasp cylindrical spindles randomly hanged in target region 2, then place them in target region 3, and return to the initial state.

A mapping from the state space to the joint space is established directly to form an IK-free trajectory generation framework. Thus, the action a_t is defined as

$$a_t = [\Delta a_1, \Delta a_2, \Delta a_3, \Delta a_4, \Delta a_5, \Delta a_6, \Delta a_7] \tag{2}$$

where $\Delta a_i (i = 1, 2, \ldots, 7)$ is the angle increment of the ith joint of the manipulator.

The state of the manipulator and the end-effector, as well as the relation between the TCP and the target objects, should be observed from the environment

$$s_t = [A_1, A_2, A_3, A_4, A_5, A_6, A_7, P_x, P_y, P_z, \Delta x, \Delta y, \Delta z, \Delta\alpha, \Delta\beta, \Delta\gamma, T] \tag{3}$$

where $A_i (i = 1, 2, \ldots, 7)$ is the angle of the ith joint at the current time step, and (P_x, P_y, P_z) is the current position of the TCP relative to the world coordinate system. $(\Delta x, \Delta y, \Delta z)$ and $(\Delta\alpha, \Delta\beta, \Delta\gamma)$ are the deviations of the position and orientation between the TCP and the spindle, respectively. T is a Boolean value which equals 1 when the TCP reaches the target region.

B. Dual-Memory Deep Deterministic Policy Gradient (DMDDPG) Algorithm

As DDPG presents excellent performance in continuous and high-dimensional action space, it is chosen in this work to handle the aforementioned multiprocess robotic tasks.

In DRL, experience replay strategy plays an important role to decrease the deviation produced in value evaluation after the functions are nonlinearly approximated. The conventional DDPG utilizes the most fundamental FIFO approach to store the experience data $D_{\mathrm{M}}^{(t)}(s_t, a_t, r_t, s_{t+1})$ in a memory buffer with a finite capacity N_{M}, and traditional data sampling considers global optima throughout the training process. However, in early training period where there is little successful experience data, it costs plenty of episodes to train the agent until it is capable to collect some successful experiences. This problem is particularly obvious for training DRL in complex tasks with random targets.

Experience data with high reward has the potential of motivating the agent to get more successful experiences in early training period through trials. Thus, a dual-memory architecture is proposed, as shown in Fig. 2. The first memory buffer follows the FIFO strategy and the second one copies and ranks all the data according to the rewards. Specifically, the second memory has two intervals: 1) interval $\boldsymbol{D}_{\mathrm{M1}}$ with a capacity of m_1 that stores the high-reward data $D_{\mathrm{M1}}^{(1)}, D_{\mathrm{M1}}^{(2)}, \ldots, D_{\mathrm{M1}}^{(m_1)}$ ranked ahead; 2) interval $\boldsymbol{D}_{\mathrm{M2}}$ with a capacity of $m_2{=}N_{\mathrm{M}}{-}m_1$ that stores the remaining low-reward data.

However, if all the N-size batch data are sampled in $\boldsymbol{D}_{\mathrm{M1}}$ throughout the training process, the algorithm can get higher success rate but is likely to be overfitted and trapped into local optima due to the large sampling bias. Thus, a local-to-global sampling strategy is proposed. As shown in Fig. 2 and Algorithm 1, the N-size batch data are composed of $(N{-}N_1)$-size and N_1-size local batches of data (denoted as $d_{m1}^{(1)}, d_{m1}^{(2)}, \ldots, d_{m1}^{(N-N_1)}$ and $d_{m2}^{(1)}, d_{m2}^{(2)}, \ldots, d_{m2}^{(N_1)}$ sampled from $\boldsymbol{D}_{\mathrm{M1}}$ and $\boldsymbol{D}_{\mathrm{M2}}$, respectively).

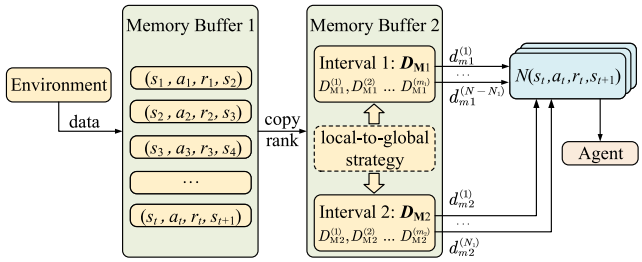


Fig. 2. Dual-memory architecture of the DMDDPG.

Algorithm 1: Implementation of the Dual-memory Architecture with Local-to-Global Strategy.

```
Initialize N_M, N, N_1, N_1max, m_1 and m_2 for the
  dual-memory architecture;
count ← 0, N_1 ← 0;
for each early training step do
    count ← count + 1;
    if count = count_max and N_1 < N_1max then
        N_1 ← N_1 + 1, count ← 0;
        if N_1 ≥ N_1max then
            N_1 ← N_1max
    Sample (N–N_1) and N_1 data from D_M1 and
      D_M2 respectively and train the networks.
```

Remark 1: To make the sampling increasingly unbiased as training proceeds, to start with, N_1=0 and it keeps increasing periodically until reaching a maximum value of $N_{1\max}$ as the training goes on. Hence, the algorithm gets more successful experiences in the early training and further transfers to global optima stably. Besides, it is preferable that $N_{1\max}/N$ is proportional to the replay capacity ratio m_2/N_{M}, in order to decrease the bias that exists in sampling.

The actor establishes a mapping from state space to action space. At state s_t, the agent executes an action a_t generated by the actor, which is defined as

$$a_t = \mu(s_t; \theta^{\mu}) + \mathcal{N}_t \tag{4}$$

where $\mu(s_t; \theta^{\mu})$ is the online policy network with parameter θ^{μ}, $\mathcal{N}_t$ is a temporally correlated Ornstein–Uhlenbeck (OU) noise that can promote the exploration efficiency of the physical system, as demonstrated in [19].

The critic will evaluate the performance of the action a generated by actor at state s and output a Q-value, denoted as $Q(s, a; \theta^{Q})$, where θ^{Q} is the parameter of the online value network.

At each training step, the agent randomly samples N-size mini-batch data tuples from the dual-memory architecture mentioned above in this section, denoted as $N(s_t, a_t, r_t, s_{t+1})$, to update the parameters θ^{μ} and θ^{Q} of the online networks of the actor and the critic.

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 25,2026 at 09:16:33 UTC from IEEE Xplore. Restrictions apply.

The online critic network is optimized with learning rate l_c by minimizing the mean-square-error defined as

$$L(\theta^Q) = \frac{1}{N}\sum_i \left(y_i - Q(s_i, a_i; \theta^Q)\right)^2 \tag{5}$$

where y_i is the target Q-value calculated by Bellman equation

$$y_i = r_i + \rho Q'\left(s_{i+1}, \mu'(s_{i+1}; \theta^{\mu'}); \theta^{Q'}\right) \tag{6}$$

where μ' is the target policy network with parameter $\theta^{\mu'}$ and Q' is the target value network with parameter $\theta^{Q'}$.

The online actor network is updated with learning rate l_a according to the expected gradient of the $Q(s, a; \theta^Q)$, which is approximate to the gradient of target function $J_\beta(\mu)$ [19]

$$J_\beta(\mu) = \mathbb{E}_\mu[r_1 + \rho r_2 + \rho^2 r_3 + \cdots + \rho^{n-1} r_n]. \tag{7}$$

As defined by (7), $J_\beta(\mu)$ is the mathematical expectation of the discounted future cumulative reward according to (1). Hence, the update approach can be obtained through calculating the derivative of the target function $J_\beta(\mu)$ by following the chain rule

$$\nabla_{\theta^\mu} J_\beta(\mu) \approx \frac{1}{N}\sum_i \left(\nabla_a Q(s, a; \theta^Q)\Big|_{s=s_i, a=\mu(s_i)} \cdot \nabla_{\theta^\mu}\mu(s; \theta^\mu)\Big|_{s=s_i}\right) \tag{8}$$

where ∇_{θ^μ} and ∇_a are the gradients for θ^μ and a, respectively.

As both the actor and critic utilize a dual-network architecture to promote the performance of the DMDDPG, while the online networks of critic and actor update the parameters according to (5) and (8), the target networks optimize the parameters using soft update

$$\begin{cases} \theta^{Q'} \leftarrow \tau\theta^Q + (1-\tau)\theta^{Q'} \\ \theta^{\mu'} \leftarrow \tau\theta^\mu + (1-\tau)\theta^{\mu'} \end{cases} \tag{9}$$

where $\tau \in [0, 1]$ is the update rate.

Besides the aforementioned dual-memory buffer, dual-network architecture, and soft update strategy, some other training skills are also adopted to make the DMDDPG more stable and converge faster:

1) The maximum permitted errors of position and orientation are set to large values and decrease as training proceeds. This dynamic criteria will help the agent to get more successful trials that feature high rewards in early training, and the policies are further optimized as the criteria become strict later.
2) The actions generated by actor are limited to $[-\delta, +\delta]$, where δ is a small constant.
3) In the critic, as dimensions of s_t and a_t are quite different, concatenating them directly and sending to the first layer is likely to bring unbalanced weight between the two parts. Thus, they are first processed separately by two different dense layers. Then, the processed information is concatenated and sent to the next hidden layer.

III. Nested-DMDDPG (N-DMDDPG) Architecture

In pick-and-place assembly and some other multiprocess scenarios of similar kind, a complete task is composed of several sequential trajectories to avoid collisions and enhance the fluency of the movement of the manipulator, and thus a novel nested architecture of multiple DMDDPGs that form a serial loop is proposed to carry out the whole multiprocess task through several adjacent trajectory segments per circle.

A. Pick-and-Place Strategy

As depicted in Fig. 1, at the beginning of the task, the manipulator stays at the initial position. The spindle is a hollow cylinder and the positioner is composed of a fixed base and a rotatable shelf (which can rotate along the z-axis) with six pegs on each side. To generalize the problem, the spindles are assumed to randomly hang along x-axis in target region 2 rather than some fixed points.

In the pick-and-place task performed in [11]–[13], both the criteria for position and orientation control in pick-and-place task are lax because the fingers will automatically correct the orientation when picking up the object. However, in our scenario and others of similar kind, strict criteria for both position and orientation are essential to avoid the collision. Besides, it is obvious that the manipulator will have to take pretty frequent changes in both position and orientation in picking up and placing the spindles.

Obviously, as shown in Fig. 1, if the manipulator directly moves from initial location to the spindles randomly hanged in target region 2 without desirable orientation, collisions will be inevitable. Hence, the whole process of the pick-and-place task has to be divided into several sequential trajectory segments by following the three steps below.

Step 1: The TCP of the manipulator moves from initial location (denoted as point A) to the preparation location (denoted as target region 1) right in front of the spindle with appropriate orientation through trajectory segment 1.

Step 2: The TCP reaches the pick-up location (denoted as target region 2), catches the spindle randomly placed in this region, and pulls it out through trajectory segment 2.

Step 3: The TCP reaches the placing location (denoted as target region 3) through trajectory segment 3, then opens the gripper, and places the spindle on the conveyor.

B. Nested Architecture of DMDDPG

As described above, the whole pick-and-place behavior is divided into three steps in time sequence. As each trajectory segment corresponding to each step is generated by different DMDDPGs, a multiprocess architecture is indispensable.

As shown in Fig. 3, unlike the multiagent structures in [23]–[25], where multiple agents are activated at the same time and execute tasks in parallel, the proposed nested architecture is a serial loop composed by several DMDDPGs and it mainly has the following features.

1) The nested architecture develops an agent with multiple policies. Each DMDDPG in the nested architecture is activated at different time steps successively to generate trajectory segments of the multiprocess task. For example,

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 25,2026 at 09:16:33 UTC from IEEE Xplore. Restrictions apply.

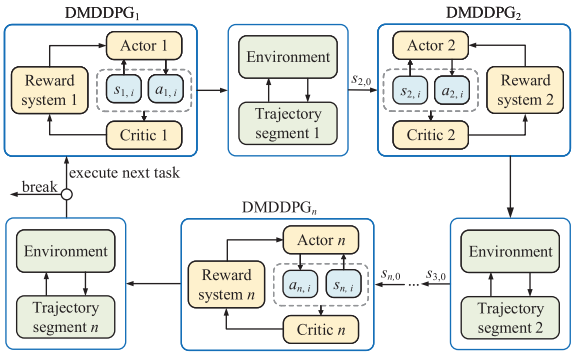


Fig. 3. Framework of the nested-DMDDPG algorithm.

the DMDDPG$_2$ begins to work only when DMDDPG$_1$ has accomplished generating trajectory segment 1.

2) Different reward functions and network structures can be designed for each DMDDPG in the nested architecture based on the following considerations: A single DMDDPG can hardly handle a multiprocess task with several subtasks. Different weights between position control and orientation control can be assigned for each step. For example, in Step 3, if the spindles are simply put in a box, the weight of orientation control can be reduced and the policies will be easier to learn.
3) The former DMDDPG in the nested architecture will provide an initial state $s_{n,0}$ for each training episode of the latter one. For instance, for each episode of DMDDPG$_2$, first, the TCP is reset to point B in region 1 (right in front of the spindle) with appropriate orientations following trajectory segment 1 generated by DMDDPG$_1$, then the current state defined by (3) is assigned as initial state $s_{2,0}$.

Remark 2: It is extremely difficult to learn each trajectory segment using three independent DMDDPGs. Take Step 2 as an example, for each episode of DMDDPG$_2$, the initial position of the TCP is randomly assigned in target region 1, and the joint angles of the manipulator should be manually solved by IK, which is complicated, model-based, and has infinite solutions for redundant manipulators. Besides, to avoid sudden changes, the joint angles should be exactly the final state of trajectory segment 1 generated by DMDDPG$_1$, which is difficult to be guaranteed. However, in the nested architecture, by using the former DMDDPG to initialize the state of the latter one, the whole assembly task avoids manually solving IK and the manipulator can transit smoothly from one trajectory segment to the next without sudden changes on joint angles. In addition, it is worth mentioning that the DMDDPGs in the nested architecture are trained in sequence rather than trained simultaneously so that the former one is stable enough to reset the environment to train the latter one.

Remark 3: As shown by Fig. 3, the nested architecture strictly obeys the following principle: The policy of generating the next trajectory segment $i+1$ is explored and learned by DMDDPG$_{i+1}$ based on the result of the current segment i. In other words, no matter what the final state of the current segment is, DMDDPG$_{i+1}$ is capable to find and learn a desired policy for generating the next segment successfully.

IV. Compound Reward System

In order to improve the robustness as well as the learning efficiency of the trajectory generation of the agent and achieve accurate position and orientation control simultaneously, in this section, the implementation of the proposed compound reward system with a weight coefficient matrix is described, which is composed of six parts.

Most of the existing applications of DRL on manipulators only focus on position control [11], [12], [16], and the reward functions are simply set as

$$r_{\text{pos1}} = -e_{\text{pos}} = -\left\|\boldsymbol{p_1} - \boldsymbol{p_0}\right\| \tag{10}$$

where $\|\cdot\|$ represents the Euclidean norm, $\boldsymbol{p_0} = (x_0, y_0, z_0)$ and $\boldsymbol{p_1} = (x_1, y_1, z_1)$ are the current position vectors of the target and the TCP, respectively, and e_{pos} is the position error.

However, for most assembly scenarios including pick-and-place task, both position and orientation of the TCP should be considered. Tait–Bryan transformation $\boldsymbol{R}(\alpha, \beta, \gamma)$ can intuitively describe orientations by rotating γ around the z-axis, β around the y-axis, and α around the x-axis of the world coordinates. Thus, an effective orientation control reward function based on Tait–Bryan angles [26] is proposed, and the second part of the compound reward can be defined as

$$r_{\text{ori1}} = -e_{\text{ori}} = -\left(|\alpha_t - \alpha_0| + |\beta_t - \beta_0| + |\gamma_t - \gamma_0|\right) \tag{11}$$

where $(\alpha_t, \beta_t, \gamma_t)$ and $(\alpha_0, \beta_0, \gamma_0)$ are the Tait–Bryan angles of the TCP and the spindles relative to the world coordinates at time t. e_{ori} is the orientation error.

Remark 4: Tait–Bryan angles have one less dimensional component compared with quaternions, go through all the three axes of cartesian coordinates, and different objects refer to the same fixed coordinate system all the time compared with proper Euler angles that have gimbal-lock problem. Thus, the orientation deviation between the TCP and the spindles defined by (11) is simpler, more intuitive, and explicit. Besides, it is very common to directly take the translation and rotation components of the TCP as action a_t, where gimbal locks occur potentially [13], [14], [16], [17]. However, as defined by (2), the joint increments Δa_i of the manipulator are taken as a_t to establish an IK-free method in the proposed framework. At each time step t, after the joints taking slight rotations, very small changes of the orientation variation of the TCP are calculated. Such trajectory generation approach is performed on the basis of joint space and can avoid gimbal lock, as it neither involves proper Euler angles nor directly rotates the TCP. By the proposed orientation control reward function, the agent tries to find solutions to make $\alpha_t{=}\alpha_0$, $\beta_t{=}\beta_0$, and $\gamma_t{=}\gamma_0$ by several successive time steps.

In order to guide the TCP to move in the direction of constantly decreasing the errors defined by (10) and (11), the third and fourth parts of the compound reward are defined as

$$r_{\text{pos2}} = \begin{cases} \phi, & \text{if } e_{\text{pos}}(n) < e_{\text{pos}}(n-1) \\ 0, & \text{if } e_{\text{pos}}(n) = e_{\text{pos}}(n-1) \\ -\phi, & \text{if } e_{\text{pos}}(n) > e_{\text{pos}}(n-1) \end{cases} \tag{12}$$

$$r_{\text{ori2}} = \begin{cases} \eta, & \text{if } e_{\text{ori}}(n) < e_{\text{ori}}(n-1) \\ 0, & \text{if } e_{\text{ori}}(n) = e_{\text{ori}}(n-1) \\ -\eta, & \text{if } e_{\text{ori}}(n) > e_{\text{ori}}(n-1) \end{cases} \tag{13}$$

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 25,2026 at 09:16:33 UTC from IEEE Xplore. Restrictions apply.

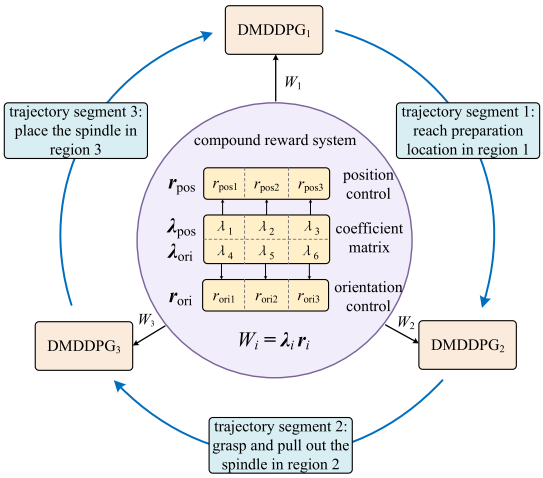


Fig. 4. Framework of N-DMDDPG combined with compound reward system.

where ϕ and η are small positive constants, and $e_{\text{pos}}(n)$ and $e_{\text{ori}}(n)$ are position and orientation errors at time step n, respectively.

Remark 5: After the TCP has reached the target, it is supposed to be kept within the maximum permitted errors for the following several training steps, where r_{pos1} and r_{ori1} are not sensitive enough to the slight changes of the errors because the variations are almost zero. In this phase, r_{pos2} and r_{ori2} can distinguish the actions that slightly reduce the errors from those slightly increase the errors.

For those steps that keep the position and orientation of the TCP within the desirable deviations (which means the task is implemented), large rewards are given as

$$r_{\text{pos3}} = \begin{cases} \varphi, & \text{if } |e_{\text{pos}}(n)| \le \epsilon_1 \\ 0, & \text{if } |e_{\text{pos}}(n)| > \epsilon_1 \end{cases} \tag{14}$$

$$r_{\text{ori3}} = \begin{cases} \xi, & \text{if } |e_{\text{ori}}(n)| \le \epsilon_2 \\ 0, & \text{if } |e_{\text{ori}}(n)| > \epsilon_2 \end{cases} \tag{15}$$

where φ and ξ are positive constants much bigger than ϕ and η. ϵ_1 and ϵ_2 is the maximum permitted position and orientation errors, respectively, which are determined by the maximum opening degree of the gripper and the size of the spindle.

To make the reward system more flexible in generating different trajectory segments, a weight coefficient matrix is adopted and the compound reward W_i for DMDDPG$_i$ is defined as

$$W_i = \boldsymbol{\lambda}_i \boldsymbol{r}_i = \left[\boldsymbol{\lambda}_{\text{pos}} |\ \boldsymbol{\lambda}_{\text{ori}}\right] \left[\boldsymbol{r}_{\text{pos}} |\ \boldsymbol{r}_{\text{ori}}\right]^{\text{T}} \tag{16}$$

where $\boldsymbol{\lambda}_{\text{pos}}=[\lambda_1\ \lambda_2\ \lambda_3]$ and $\boldsymbol{\lambda}_{\text{ori}}=[\lambda_4\ \lambda_5\ \lambda_6]$ are coefficient matrices for DMDDPG$_i$. $\boldsymbol{r}_{\text{pos}}=[r_{\text{pos1}}\ r_{\text{pos2}}\ r_{\text{pos3}}]$ and $\boldsymbol{r}_{\text{ori}}=[r_{\text{ori1}}\ r_{\text{ori2}}\ r_{\text{ori3}}]$ are the position control and orientation control matrices, respectively.

Remark 6: As shown in Fig. 4 and Algorithm 2, combined with compound reward system, the proposed N-DMDDPG is capable to cater for the needs of tuning different weights between position control and orientation control when generating different trajectory segments. For example, for simple pick-and-place task that does not require accurate orientation of the TCP, the elements of $\boldsymbol{\lambda}_{\text{ori}}$ can be set to zero or small values to accelerate the efficiency on policy learning.

Algorithm 2: Implementation of Generating Trajectory Segments Successively using N-DMDDPG.

Initialize θ_i^Q and θ_i^μ $(i=1,2,\cdots,n)$ for DMDDPG$_i$;
$\theta_i^{Q'} \leftarrow \theta_i^Q$, $\theta_i^{\mu'} \leftarrow \theta_i^\mu$;
Initialize dual-memory buffer and weight coefficient matrix $\boldsymbol{\lambda}_i$ for DMDDPG$_i$;
for $episode_n=1,2,\cdots,M$ **do**
  for $episode_{n-1}=1,2,\cdots,J$ **do**
    $\cdots$
    for $episode_1=1,2,\cdots,K$ **do**
      Receive initial state $s_{1,0}$ and generate trajectory segment 1 by DMDDPG$_1$;
      if $e_{\text{pos}}\le\epsilon_1$ and $e_{\text{orient}}\le\epsilon_2$ **then**
        $s_{2,0}\leftarrow s_{1,k}$ $(k\le K)$ and break
    $\cdots$
    Generate trajectory segment $n-1$ by DMDDPG$_{n-1}$;
    if $e_{\text{pos}}\le\epsilon_1$ and $e_{\text{orient}}\le\epsilon_2$ **then**
      $s_{n,0}\leftarrow s_{n-1,j}$ $(j\le J)$ and break
  Generate trajectory segment n by DMDDPG$_n$;
  if $e_{\text{pos}}\le\epsilon_1$ and $e_{\text{orient}}\le\epsilon_2$ **then**
    break

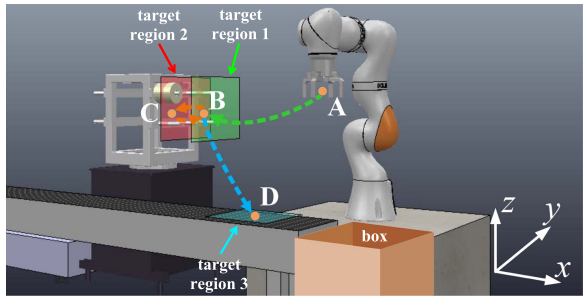


Fig. 5. Virtual twin system of the robotic pick-and-place assembly platform. A is the initial position of the tool center point (TCP). B, C, and D are random points in target regions 1, 2, and 3, respectively.

V. Experimental Results

In this section, we present the experimental setups and results to demonstrate the feasibility and performance of the proposed N-DMDDPG in multiprocess robotic tasks.

A. Experimental Setups

1) Platform Setups: The components of the whole assembly line system are shown in Figs. 1 and 5; our aim is to train the proposed N-DMDDPG algorithm to direct the 7-DoF serial manipulator KUKA LBR iiwa 7 R800 to pick up the spindle randomly hanged in target region 2 of the positioner, and then place it in target region 3, which is divided into three steps as illustrated in Section III.

As shown in Fig. 5, to avoid formulizing models, retraining on real manipulator, or sim-to-real transfer [17], [27], a VTS is built on the CoppeliaSim to make the training reliable and efficient, so as to bridge the gap between the proposed theoretical work

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 25,2026 at 09:16:33 UTC from IEEE Xplore. Restrictions apply.

TABLE I
TRAINING HYPERPARAMETERS

Parameter	Value	Parameter	Value
l_a	0.001	N_M	30000
l_c	0.001	N	32
τ	0.01	ρ	0.98

TABLE II
POSITION AND ORIENTATION REWARD PARAMETERS

DMDDPG$_i$	Reward					Coefficient				
	η	ξ	ϕ	φ	λ_1	λ_2	λ_3	λ_4	λ_5	λ_6
$i=1$	0.05	1	0.05	1	1	1	1	0.5	0.4	0.5
$i=2$	0.05	1	0.05	1	1	1	1	0.5	0.4	0.5
$i=3$	0.05	1	0.05	1	1	1	1	0.2	0.8	0.5

and its practical applications. Though collisions may happen in VTS before the algorithm is converged, no physical damage will occur. Thus, no complicated safety constraints are required. Besides, data for training are quite easy to collect and it is convenient to initialize the scenario after each training episode. Bullet 2.83 is chosen as the physics engine of VTS, and the simulation time step is set as 50 ms.

Remark 7: It is worth mentioning that the policies generated in simulation can be fed to real manipulator directly without any redundant efforts, as the VTS provides an accurate simulated environment that is 1:1 copied from the realistic scenario. Hence, the VTS can provide the learned policies to the real world with much higher efficiency and can be directly used by the trajectory planner of the pick-and-place task.

2) Parameter Setups: The hyperparameters of the training process are given in Table I, note that all the DRL algorithms use the same hyperparameters unless otherwise mentioned. The reward values and coefficients defined by (16) are shown in Table II, where the position coefficients λ_1, λ_2, and λ_3 are set as 1, and the importance of orientation is determined by tuning λ_4 relative to λ_1, λ_5 relative to λ_2 and λ_6 relative to λ_3, respectively. At the beginning, the manipulator stays at the initial position with its joint angles set as $[0, 0, 0, -90°, 0, 90°, 0]$. To make the continuous actions generated by actor stable, δ is set as 1.5°. In addition, OU noises in range of $[-0.5, 0.5]$ are added to the actions of the N-DMDDPG to extend the exploration in the training process. For PER, the priority degree, the positive constant in the priority, and the importance of PER are set as 0.6, 0.01, and 0.4, respectively. Moreover, the dynamic criteria for the maximum position error ϵ_1 and maximum orientation error ϵ_2 are set as follows: for the first 90 000 training steps, ϵ_1 and ϵ_2 are set to 5 cm and 25°, respectively. The two criteria decrease to 4 cm and 20° from the 90 000 to 120 000 training steps, then decrease to 3 cm and 15° in the next 30 000 training steps, and further decrease to 2 cm and 10° in the following training steps. For the local-to-global strategy, $N_{1\max}$, $count_{\max}$, m_1, and m_2 are set as 12, 8000, 20 000, and 10 000, respectively.

B. Network Training and Contribution Verification

In this part, the performance of the DMDDPG is evaluated in comparison to the traditional DDPG [27] on a 2-DoF manipulator and the 7-DoF serial manipulator.

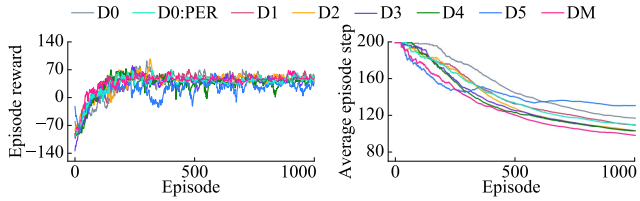


Fig. 6. Performance of DMDDPG compared with DDPG on a 2-DoF manipulator. D0 and DM represent the traditional DDPG and DMDDPG with local-to-global strategy. D0:PER denotes DDPG with PER. D1–D5 are DMDDPGs trained only by the high-reward interval $\boldsymbol{D}_{M1}$ throughout the time, with m_1 set as 28 000, 26 000, 22 000, 20 000, and 10 000, respectively.

TABLE III
SUCCUSS RATE OF DIFFERENT ALGORITHMS IN DIFFERENT PERIODS

Algos	Training episode				Testing episode
	1-250	251-500	501-750	751-1000	1-500
D0	18.4%	74.4%	89.2%	89.6%	93.2%
D1	26.8%	85.6%	85.2%	95.6%	90.2%
D2	28.0%	90.8%	93.2%	94.8%	91.4%
D3	35.2%	88.0%	92.0%	96.4%	91.2%
D4	39.2%	90.0%	91.2%	94.4%	90.8%
D5	49.2%	58.8%	62.0%	73.2%	57.6%
D0:PER	30.8%	82.8%	89.6%	93.2%	94.2%
DM	46.4%	87.6%	96.0%	94.8%	95.0%

First, the effect of the proposed dual-memory architecture and local-to-global strategy is evaluated on a 2-DoF manipulator. The task is to reach the random targets in its workspace and the training has run 1000 episodes with a max step of 200. As shown by Fig. 6 and the success rate of different periods (defined as the ratio of success numbers to interval size) in Table III, high-reward data in $\boldsymbol{D}_{M1}$ has the potential to motivate the algorithms to get more successful experiences in the early training period, and thus they can get higher rewards and converge faster than traditional DDPG (D0). However, if the algorithms are trained only by high-reward data throughout the time (see D1–D5), they are likely to be trapped in local optima so that they get lower success rate than D0 in testing process. Besides, if the capacity m_1 of the $\boldsymbol{D}_{M1}$ is too small (see D5), the algorithm shows poor stability and low success rate in both training and testing processes as the diversity of the data samples is restricted. Dramatically, compared with D1 to D5 and DDPG with PER (D0:PER), by carrying out the DMDDPG with local-to-global strategy, the algorithm (DM) not only converges faster, but also keeps a relatively higher success rate in both the training and testing processes. In addition, take training episodes from 500 to 1000 as examples, the standard deviations (SD) of the reward of DM, D0, and D0:PER are 19.04, 28.34, and 28.17, respectively. Thus, the reward variation of DM is much more smooth and stable, while other algorithms still have apparent fluctuations.

Second, as is known, the complexity of the neural network will directly influence its performance. Usually, a multi-hidden-layer network is more effective than a single-hidden-layer network, but it also greatly increases the computation load. To evaluate the performance of the proposed dual-memory architecture against different neural network structures as well as the feasibility of the proposed nested architecture in executing multiprocess tasks like pick-and-place assembly, each algorithm in the nested

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 25,2026 at 09:16:33 UTC from IEEE Xplore. Restrictions apply.

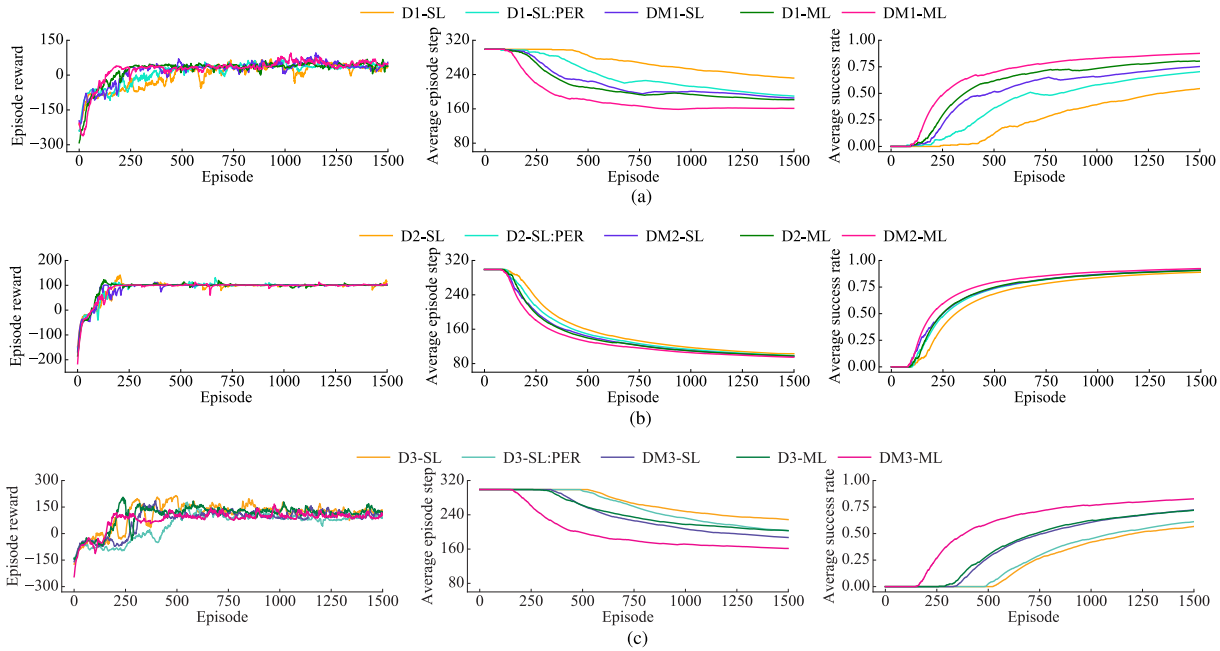


Fig. 7. Performances of single-hidden-layer DMDDPG$_i$ (DMi-SL), multi-hidden-layer DMDDPG$_i$ (DMi-ML), single-hidden-layer DDPG$_i$ (Di-SL), single-hidden-layer DDPG$_i$ (Di-SL), single-hidden-layer DDPG$_i$ with PER (Di-SL:PER), and multi-hidden-layer DDPG$_i$ (Di-ML) in generating sequential trajectory segments for the 7-DoF manipulator by the nested architecture. (a) Different algorithms for trajectory segment 1. (b) Different algorithms for trajectory segment 2. (c) Different algorithms for trajectory segment 3.

architecture has been trained 1500 episodes with a max step of 300 in the VTS on the 7-DoF serial manipulator.

As shown in Fig. 7, generally, in each subprocess, multi-hidden-layer DDPGs with dual-memory architecture (see DM1-ML, DM2-ML, and DM3-ML) converge faster, take less steps in executing tasks, and gain higher success rates; they are more efficient, more stable, and have better action strategy in training process. Also, the reward variations of these algorithms are more smooth without large fluctuations. Single-hidden-layer DDPGs with dual-memory architecture (see DM1-SL, DM2-SL, and DM3-SL) have similar performances with multi-hidden-layer DDPGs (see D1-ML, D2-ML, and D3-ML). Specifically, single-hidden-layer DDPGs (see D1-SL, D2-SL, and D3-SL) perform worse and are even unstable in Step 1 and Step 3, and apparent fluctuations can be observed in rewards. Besides, as DMDDPGs converge faster than DDPGs and DDPGs with PER, training process for each trajectory segment is more efficient, and thus the total training time of the nested architecture is greatly saved.

Remark 8: It can be observed from Fig. 7 that by employing the proposed dual-memory architecture, the improvements of the single-hidden-layer DDPGs (with 300 nodes) are greater than that of the multi-hidden-layer DDPGs (with 64 nodes for each hidden layer in actors, 256 nodes for the first hidden layer, and 64 nodes for the second in critics). Thus, it can be deduced that the simpler the networks of the algorithms are, the better effects on promoting the performance can be achieved. This conclusion has also been verified in [16].

Remark 9: By comparing single-hidden-layer DMDDPGs (DM1-SL, DM2-SL, and DM3-SL) with single-hidden-layer DDPGs (D1-SL, D2-SL, and D3-SL) and multi-hidden-layer DDPGs (D1-ML, D2-ML, and D3-ML), it can be observed from Fig. 7 that the proposed dual-memory architecture can greatly improve a single-hidden-layer DDPG with poor performance (D1-SL, for example), and makes it good enough to be close to a multi-hidden-layer DDPG (D1-ML). However, as DM1-SL has much fewer network parameters, it relieves the computation load to a large extent when training networks.

Additionally, to demonstrate the robustness of the proposed N-DMDDPG in presence of other diverse noise disturbances compared with N-DDPG, 100 episodes have been tested for each algorithm under three different circumstances: 1) The algorithms are tested under the original OU noise in range of $[-0.5, 0.5]$; 2) the original OU noise is replaced by a larger OU noise in range of $[-1, 1]$ (named as Di-ML:OU and DMi-ML:OU); 3) the OU noise is replaced by a Gaussian noise with the variance set as 0.5 (named as Di-ML:GS and DMi-ML:GS). As shown in Table IV, Di-ML:GS and DMi-ML:GS keep similar average steps with the original algorithms. Specifically, Di-ML:OU and DMi-ML:OU tend to take much more steps to implement tasks, and have larger SD than the original algorithms, which indicates larger fluctuations of the episode steps around the average value. Moreover, it can be observed that the whole performances of DMDDPGs are more robust than DDPGs. Thus, it can be deduced that the proposed N-DMDDPG shows quite excellent stability against different noise disturbances and is fully qualified to implement the multiprocess assembly.

C. Application Instances

In this part, as shown in Fig. 1, physical experiments are presented to evaluate the effect of the proposed compound reward system with both position and orientation control, and the feasibility of generating sequential trajectory segments for multiprocess robotic tasks by the proposed N-DMDDPG.

First, the feasibility and effect of the proposed compound reward system with both position and orientation control are verified through 50 tests. As shown in Fig. 8, if trajectory segment 1 is generated by DMDDPG$_1$ with only position control

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 25,2026 at 09:16:33 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
PERFORMANCES OF N-DDPG AND N-DMDDPG AGAINST DIFFERENT NOISE DISTURBANCES

	D*i*-ML / DM*i*-ML		D*i*-ML:GS / DM*i*-ML:GS		D*i*-ML:OU / DM*i*-ML:OU	
	average episode step	SD	average episode step	SD	average episode step	SD
$i=1$	123.8 / 109.8	18.0 / 13.3	132.8 / 106.2	7.8 / 8.1	205.9 / 121.4	55.0 / 21.5
$i=2$	69.3 / 68.8	8.2 / 7.5	68.1 / 67.6	2.8 / 2.8	81.65 / 79.3	17.2 / 14.4
$i=3$	133.6 / 129.4	31.0 / 30.2	128.9 / 128.5	24.9 / 25.7	155.6 / 135.2	45.0 / 37.7

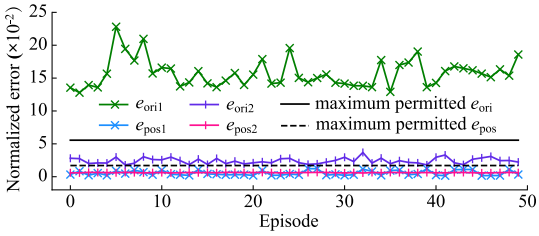


Fig. 8. Comparison of generating trajectory 1 without orientation control (see e_{pos1} and e_{ori1}) and with orientation control (see e_{pos2} and e_{ori2}).

TABLE V
COMPARISON OF PLACING THE SPINDLE INTO THE BOX

Items	Coefficient matrices			
	$\boldsymbol{\lambda}_a$	$\boldsymbol{\lambda}_b$	$\boldsymbol{\lambda}_c$	$\boldsymbol{\lambda}_d$
episode of being convergent	675	225	349	293
$e_{\mathrm{pos}}(\times 10^{-2})$	1.52	0.91	0.73	0.96
$e_{\mathrm{ori}}(\times 10^{-2})$	0.06	39.44	0.31	0.09
trajectory length (m)	1.49	1.44	2.19	1.67
task execution time (s)	21.98	18.77	31.30	24.40

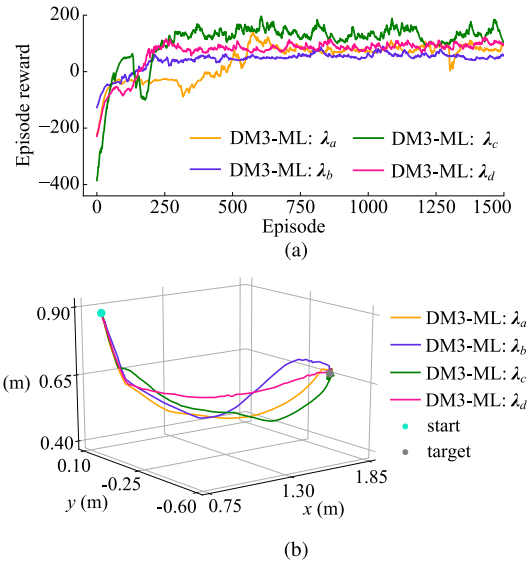


Fig. 9. Performance of placing the spindle into the box by DMDDPG trained with different reward coefficients, where $\boldsymbol{\lambda}_a = [0.2, 0.4, 0.5, 1, 1, 1.5]$, $\boldsymbol{\lambda}_b = [1, 1, 1, 0, 0, 0]$, $\boldsymbol{\lambda}_c = [1, 1, 1, 1, 1, 1]$, and $\boldsymbol{\lambda}_d = [1, 1, 1, 0.2, 0.8, 0.5]$. (a) Reward variations in the training. (b) Task trajectory examples.

$\boldsymbol{r}_{\mathrm{pos}}$, it can reach the random target points with small position errors (see e_{pos1}). However, considerable orientation errors (see e_{ori1}) can be observed. Nevertheless, by importing orientation control $\boldsymbol{r}_{\mathrm{ori}}$ based on Tait–Bryan angles, DMDDPG$_1$ can not only reach the random target points with small position errors (see e_{pos2}), but also keep the orientation error (see e_{ori2}) in a desirable range, which makes it qualified to implement both position and orientation control when generating trajectories using DRL methods. Obviously, if the TCP does not reach region 1 with a desired orientation in Step 1, then collision may happen in Step 2 because the orientation cannot be adjusted in time when the gripper is approaching the spindle. Thus, in both the two steps, accurate orientation control must be considered. However, in Step 3, the weight of orientation control can be adjusted according to the actual demands of the subtask.

To verify the effect of the coefficient matrix, as shown by Fig. 5, the third subtask (Step 3) is replaced by putting the spindle into a box, and DM3-ML is trained with different coefficient matrices. All the algorithms can handle this subtask and Fig. 9(b) presents some examples. However, there are some differences among the trajectories generated by different coefficient matrices. As shown in Table V and Fig. 9(a), in $\boldsymbol{\lambda}_a$, the importance of orientation is much bigger than that of position, and DM3-ML:$\boldsymbol{\lambda}_a$ implements the task with the smallest e_{ori} but the largest e_{pos}. It is also the slowest one to be convergent in the training. In $\boldsymbol{\lambda}_b$, only the position is controlled and DM3-ML:$\boldsymbol{\lambda}_b$ has the fastest convergent rate in the training as the policy is much easier to be found and learned. It can also generate task trajectories with the shortest length and shortest execution time. In $\boldsymbol{\lambda}_c$, the orientation and position have the same importance, and DM3-ML:$\boldsymbol{\lambda}_c$ achieves both accurate control on the two aspects. However, it generates trajectory with the longest length and longest execution time, and thus it is not efficient enough. $\boldsymbol{\lambda}_d$ is the fine-tuned coefficient for the original Step 3, and DM3-ML:$\boldsymbol{\lambda}_d$ can implement the new subtask with both small e_{pos} and small e_{ori}, but the trajectory tends to have longer length and more execution time.

Hence, if the new subtask does not have a demand for the orientation, then DM3-ML:$\boldsymbol{\lambda}_b$ is the most efficient one for this subtask with the lowest costs on both the training and executing. If the the spindle is required to be placed with an appropriate orientation and fewer execution time, then DM3-ML:$\boldsymbol{\lambda}_d$ is preferable. To conclude, compared with other reward systems like fuzzy reward in [17] that needs to rebuild all the fuzzy rules to adjust the weight, the proposed compound reward with coefficient matrix can flexibly balance the weight between the orientation control and the position control, to cater for the need of different robotic tasks based on DRL.

Second, as shown in Fig. 10, to verify the robustness of the proposed algorithm against new target positions or orientations, each situation has been tested 50 episodes. It can be seen that DMDDPG$_1$ against new positions (see e_{ori1} and e_{pos1}) can keep desirable performances with very small errors and fluctuations. For DMDDPG$_1$ against new orientations (see e_{ori2} and e_{pos2}), it

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 25,2026 at 09:16:33 UTC from IEEE Xplore. Restrictions apply.

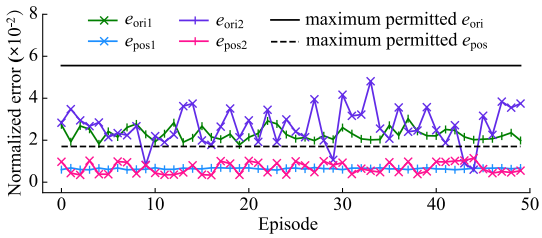


Fig. 10. Performances of DMDDPG$_1$ against new positions (random points out of the region 1 within 7 cm while keeping the orientation unchanged (see e_{ori1} and e_{pos1}) and new orientations (randomly rotate α and β of the points in region 1 within $[-10^\circ, 10^\circ]$; see e_{ori2} and e_{pos2}).

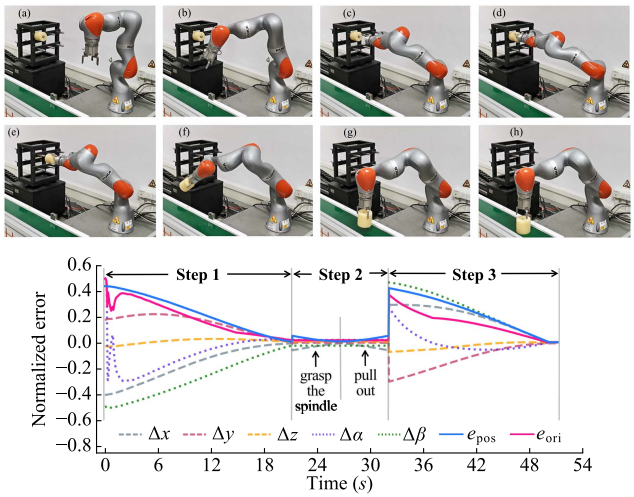


Fig. 11. Error variations of the pick-and-place assembly on realistic platform. (a) start at initial location A. (b) move along trajectory 1. (c) reach target region 1. (d) move along trajectory 2. (e) pull out the spindle in region 2. (f) move along trajectory 3. (g) reach target region 3. (h) place the spindle.

can still implement the task with relatively small errors though the errors have more fluctuations and are larger than e_{ori1} and e_{pos1}.

Finally, to present a complete process of generating sequential trajectory segments for multiprocess tasks using the proposed N-DMDDPG, the following multiprocess robotic task is taken as an example: The spindle in the upper left corner is picked up from the positioner and then placed on the conveyor. The close-up sequence and the variations of the normalized errors defined by (10) and (11) of the whole assembly scheme are presented in Fig. 11, where (a)–(c), (d)–(e), and (f)–(h) are for Step 1, Step 2, and Step 3, respectively. It can be observed that, by feeding the joint angle sequences offered by N-DMDDPG to the realistic manipulator, the TCP follows the three trajectory segments generated in the VTS successively. First, the TCP reaches the preparation location in target region 1 with accurate position and desirable orientation. Then, as the orientation is already well prepared in Step 1, the TCP only needs to move straight along the x-axis to target region 2, and the orientation error does not experience a great change in grasping and pulling out the spindle with only changing Δx. Finally in Step 3, both the position and the orientation change frequently and the DMDDPG$_3$ presents good performance on driving the TCP to target region 3 by decreasing the errors. Hence, it can be verified that the proposed N-DMDDPG can handle trajectory generation problem for multiprocess robotic tasks well.

Remark 10: Similar to the implementation of the aforementioned assembly task, spindles that randomly appear in target region 2 can be picked up and placed with the three trajectory segments generated in the VTS by the proposed N-DMDDPG. Besides, the proposed N-DMDDPG is actually a general method that is qualified to handle other multiprocess scenarios of the similar kind, as well as more complex tasks by increasing the DMDDPGs in the nested architecture.

VI. Conclusion

This article discusses the trajectory generation problem for multiprocess robotic tasks using N-DMDDPG algorithm. We have shown that the proposed N-DMDDPG develops a multipolicy agent through the nested architecture. It can generate sequential trajectory segments asynchronously in one scheme and avoid complicated IK calculations, where a dual-memory architecture and local-to-global strategy are adopted to enhance the performance of each DMDDPG. A compound reward system is proposed to achieve both position control and orientation control simultaneously, where a novel orientation control method based on Tait–Bryan angles is designed. In addition, a VTS is constructed to directly apply the simulated policies to real-world platform without any sim-to-real transfer. This work clearly states the design details of the N-DMDDPG and its feasibility in multiprocess robotic tasks.

The proposed N-DMDDPG scheme is evaluated on a 2-DoF planar manipulator and a 7-DoF serial manipulator. The experimental results demonstrate that the proposed DMDDPG is more stable and effective, converges faster, and can get higher success rate in training than DDPG. In addition, it shows good robustness in presence of diverse noise disturbances. Besides, the experimental results show that the trained agent is robust to new positions or orientations. Actually, the proposed N-DMDDPG is a general approach that can be applied to other multiprocess scenarios and manipulators with arbitrary DoFs. Additionally, retraining is still required if the well-learned algorithm is applied to new workspace. To address this limitation, the proposed scheme can be further studied by referring to the advantages of transfer learning.

References

[1] F. Wang, X. Zhou, J. Wang, X. Zhang, Z. He, and B. Song, "Joining force of human muscular task planning with robot robust and delicate manipulation for programming by demonstration," *IEEE/ASME Trans. Mechatronics*, vol. 25, no. 5, pp. 2574–2584, Oct. 2020.

[2] H. Liu, X. Lai, and W. Wu, "Time-optimal and jerk-continuous trajectory planning for robot manipulators with kinematic constraints," *Robot. Comput.-Integr. Manuf.*, vol. 29, no. 2, pp. 309–317, 2013.

[3] Z. Chen, F. Huang, W. Sun, J. Gu, and B. Yao, "RBF-neural-network-based adaptive robust control for nonlinear bilateral teleoperation manipulators with uncertainty and time delay," *IEEE/ASME Trans. Mechatronics*, vol. 25, no. 2, pp. 906–918, Apr. 2020.

[4] P. Acharya, K.-D. Nguyen, H. M. La, D. Liu, and I.-M. Chen, "Nonprehensile manipulation: A trajectory-planning perspective," *IEEE/ASME Trans. Mechatronics*, vol. 26, no. 1, pp. 527–538, Feb. 2021.

[5] B. Brahmi, M. Saad, M. H. Rahman, and C. Ochoa-Luna, "Cartesian trajectory tracking of a 7-DoF exoskeleton robot based on human inverse kinematics," *IEEE Trans. Syst., Man, Cybern.: Syst.*, vol. 49, no. 3, pp. 600–611, Mar. 2019.

[6] T. George Thuruthel, Y. Ansari, E. Falotico, and C. Laschi, "Control strategies for soft robotic manipulators: A survey," *Soft Robot.*, vol. 5, pp. 149–163, 2018.

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 25,2026 at 09:16:33 UTC from IEEE Xplore. Restrictions apply.

[7] C. Guarinolobianco, M. Faroni, M. Beschi, and A. Visioli, "A predictive technique for the real-time trajectory scaling under high-order constraints," *IEEE/ASME Trans. Mechatronics*, vol. 27, no. 1, pp. 315–326, Feb. 2022, doi: 10.1109/TMECH.2021.3063627.
[8] J. P. Wang, Y. K. Shi, W. S. Zhang, I. Thomas, and S. H. Duan, "Multitask policy adversarial learning for human-level control with large state spaces," *IEEE Trans. Ind. Inform.*, vol. 15, no. 4, pp. 2395–2404, Apr. 2019.
[9] K. Chatzilygeroudis, V. Vassiliades, F. Stulp, S. Calinon, and J. Mouret, "A survey on policy search algorithms for learning robot controllers in a handful of trials," *IEEE Trans. Robot.*, vol. 36, no. 2, pp. 328–347, Apr. 2020.
[10] X. Li, H. Liu, and M. Dong, "A general framework of motion planning for redundant robot manipulator based on deep reinforcement learning," *IEEE Trans. Ind. Inform.*, 2021, to be published, doi: 10.1109/TII.2021.3125447.
[11] Y. Deng *et al.*, "Deep reinforcement learning for robotic pushing and picking in cluttered environment," in *Proc. IEEE Int. Conf. Intell. Robot. Syst.*, 2019, pp. 619–626.
[12] A. Zeng, S. Song, S. Welker, J. Lee, A. Rodriguez, and T. A. Funkhouser, "Learning synergies between pushing and grasping with self-supervised deep reinforcement learning," *IEEE Int. Conf. Intell. Rob. Syst.*, pp. 4238–4245, 2018.
[13] M. Breyer, F. Furrer, T. Novkovic, R. Siegwart, and J. Nieto, "Comparing task simplifications to learn closed-loop object picking using deep reinforcement learning," *IEEE Robot. Autom. Lett.*, vol. 4, no. 2, pp. 1549–1556, Apr. 2019.
[14] S. Chen, D. Yan, Y. Zhang, Y. Tan, and W. Wang, "Live working manipulator control model based on DPPO-DQN combined algorithm," in *Proc. Adv. Inf. Tech. Electron. Autom. Control Conf.*, 2019, pp. 2620–2624.
[15] G. Sartoretti, W. Paivine, Y. Shi, Y. Wu, and H. Choset, "Distributed learning of decentralized control policies for articulated mobile robots," *IEEE Trans. Robot.*, vol. 35, no. 5, pp. 1109–1122, Oct. 2019.
[16] A. R. Mahmood, D. Korenkevych, G. Vasan, W. Ma, and J. Bergstra, "Benchmarking reinforcement learning algorithms on real-world robots," in *Proc. 2nd Annu. Conf. Robot Learn.*, 2018, pp. 561–591.
[17] J. Xu, Z. Hou, W. Wang, B. Xu, K. Zhang, and K. Chen, "Feedback deep deterministic policy gradient with fuzzy reward for robotic multiple peg-in-hole assembly tasks," *IEEE Trans. Ind. Inform.*, vol. 15, no. 3, pp. 1658–1667, Mar. 2019.
[18] N. Naughton, J. Sun, A. Tekinalp, T. Parthasarathy, G. Chowdhary, and M. Gazzola, "Elastica: A compliant mechanics environment for soft robotic control," *IEEE Robot. Autom. Lett.*, vol. 6, no. 2, pp. 3389–3396, Apr. 2021.
[19] T. Lillicrap *et al.*, "Continuous control with deep reinforcement learning," *Int. Conf. Learn. Represent.*, pp. 1–14, 2016.
[20] Y. Li, "Deep reinforcement learning: An overview," 2017, *arxiv.org/abs/1701.07274*.
[21] D. Horgan *et al.*, "Distributed prioritized experience replay," *Int. Conf. Learn. Represent.*, pp. 1–19, 2018.
[22] T. T. Nguyen, N. D. Nguyen, and S. Nahavandi, "Deep reinforcement learning for multi-agent systems: A review of challenges, solutions, and applications," *IEEE Trans. Cybern.*, vol. 50, no. 9, pp. 3826–3839, Sep. 2020.
[23] Z. Li and C. Guo, "Multi-agent deep reinforcement learning based spectrum allocation for D2D underlay communications," *IEEE Trans. Veh. Technol.*, vol. 69, no. 2, pp. 1828–1840, Feb. 2020.
[24] X. Hu *et al.*, "Multi-agent deep reinforcement learning-based flexible satellite payload for mobile terminals," *IEEE Trans. Veh. Technol.*, vol. 69, no. 9, pp. 9849–9865, Sep. 2020.
[25] W. Liu, S. Liu, J. Cao, Q. Wang, X. Lang, and Y. T. Liu, "Learning communication for cooperation in dynamic agent-number environment," *IEEE/ASME Trans. Mechatronics*, vol. 26, no. 4, pp. 1846–1857, Aug. 2021, doi: 10.1109/TMECH.2021.3076080.
[26] E. Reyes-Valeria, R. Enriquez-Caldera, S. Camacho, and J. Guichard, "LQR control for a quadrotor using unit quaternions: Modeling and simulation," in *Proc. Int. Conf. Electron. Commun. Comput.*, 2013, pp. 172–178.
[27] Y. L. Kim, K. H. Ahn, and J. B. Song, "Reinforcement learning based on movement primitives for contact tasks," *Robot. Comput.-Integr. Manuf.*, vol. 62, pp. 101863–101863, 2020.



Fengkang Ying received the B.E. degree in electrical engineering and automation from Huaqiao University, Xiamen, China, in 2019. He is currently working toward the M.E. degree in electrical engineering with the College of Information Science and Technology, Donghua University, Shanghai, China.

Since September 2019, he has been with the College of Information Science and Technology, Donghua University. His research interests include robotics, intelligent control, and reinforcement learning.



Huashan Liu (Member, IEEE) received the B.E. degree in mechanical engineering from Wuhan University, Wuhan, China, in 2005, and the Ph.D. degree in mechatronics from Zhejiang University, Hangzhou, China, in 2010.

Since 2010, he has been with the College of Information Science and Technology, Donghua University, Shanghai, China, where he is currently an Associate Professor and the Vice Dean. From 2015 to 2016, he was a Visiting Professor with the Institute of Automatic Control Engineering, Technical University of Munich, Munich, Germany. His research interests include robotics, artificial intelligence, mechatronics, and motion control.



Rongxin Jiang received the B.E. degree in electrical engineering and automation, in 2020, from Donghua University, Shanghai, China, where he is currently working toward the M.E. degree in electrical engineering with the College of Information Science and Technology.

Since September 2020, he has been with the College of Information Science and Technology, Donghua University. His research interests include robotics, machine learning, and intelligent control.



Xin Yin is currently working toward the B.E. degree in automation with the College of Information Science and Technology, Donghua University, Shanghai, China.

She has been with the College of Information Science and Technology, Donghua University, Shanghai, China, since September 2018. Her research interests include machine vision and reinforcement learning.

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 25,2026 at 09:16:33 UTC from IEEE Xplore. Restrictions apply.